



Universidad
César Vallejo

Código fuente

**Modelo Automatizado de Patrones en
Imágenes Mamarias**



MAPIM



Índice_

Figura 1. Frame de logueo .	3
Figura 2. Código del botón acceder .	4
Figura 3. Frame de creación de usuario .	5
Figura 4. Código para guardar el usuario y contraseña en BD. .	6
Figura 5. Frame de Inicio. .	7
Figura 6. Frame de registro del pacientes. .	8
Figura 7. Código para ingresar los datos y registrar al paciente. .	9
Figura 8. Código para guardar al paciente .	10
Figura 9. Frame Detección. .	11
Figura 10. Código para generar el resultado del análisis de la imagen. .	12
Figura 11. Código para guardar resultado. .	13
Figura 12. Frame Historial .	14
Figura 13. Código para generar el historial. .	15



APLICACIÓN MAPIM

Código fuente

La figura 1 muestra el frame de logueo donde el usuario escribirá su usuario y contraseña para ingresar. Al no tener usuario se da una opción de registrarse, donde lo llevará a otro frame para crear su usuario, contraseña y registrar sus nombres y apellidos.

Iniciar Sesión

Correo Electronico

Contraseña

Entrar

¿Aún no tienes una cuenta?

Regístrate para que puedas iniciar sesión

Regístrate

Figura 1 Frame de logueo



En la figura 2, se muestra el código del botón de acceder, en esta se crea una función para poder realizar el ingreso al frame de presentación mediante un usuario y contraseña.

```
<!DOCTYPE html>
<html lang="en">
<head>
  {% load static %}
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login y Registro</title>
  <link href="https://fonts.googleapis.com/css2?family=Roboto:ital,wght@0,100;0,300;0,400;0,500;0,700;0,900;1,100;1,300;1,400;1,500;1,700;1,900&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="{% static 'css/login.css' %}">
</head>
<body>
  <main>
    <div class="contenedor_todo">
      <div class="caja_trasera">
        <div class="caja_trasera-login">
          <h3>¿Ya tienes una cuenta?</h3>
          <p>Inicia sesión para entrar en la página</p>
          <button id="btn_iniciar-sesion">Iniciar Sesión</button>
        </div>
        <div class="caja_trasera-register">
          <h3>¿Aún no tienes una cuenta?</h3>
          <p>Regístrate para que puedas iniciar sesión</p>
          <button id="btn_registrarse">Registrarse</button>
        </div>
      </div>
      <!-- Formulario de login y registro -->
      <div class="contenedor_login-register">
        <!-- Login -->
        <form action="" class="formulario_login" id="loginForm">
          <h2>Iniciar Sesión</h2>
          <input type="text" placeholder="Correo Electronico" id="correoLogin">
          <input type="password" placeholder="Contraseña" id="contraseñaLogin">
          <button type="button" id="loginButton">Entrar</button>
        </form>
      </div>
    </div>
  </main>
</body>
</html>
```

Figura 2. Código del botón acceder



En la figura 3, se muestra el frame donde se creará un nuevo usuario. Se solicita los nombres completos, correo, usuario y contraseña. Al lado se encuentra el iniciar sesión.

The image shows a user registration interface. On the left, a light blue box contains the text '¿Ya tienes una cuenta?' and 'Inicia sesión para entrar en la página' with a dark blue 'Iniciar Sesión' button. On the right, a white box titled 'Regístrate' contains four input fields: 'Nombre Completo', 'Correo Electronico', 'Usuario', and 'Contraseña', followed by a dark blue 'Regístrate' button. The background features a blue and green geometric pattern.

Figura 3. Frame de creación de usuario



En la figura 4, se muestra la lógica del procedimiento para generar el usuario con su contraseña la cual será guardada en la base de datos (BD).

```
</form>
<!-- Registro -->
<form action="" class="formulario_register" id="registerForm">
  <h2>Regístrase</h2>
  <input type="text" placeholder="Nombre Completo" id="nombreCompleto" pattern="[A-Za-zÁÉÍÓÚáéíóúñ\s]+" title="Solo se permiten letras y espacios">
  <input type="text" placeholder="Correo Electronico" id="correo">
  <input type="text" placeholder="Usuario" id="usuario">
  <input type="password" placeholder="Contraseña" id="contraseña">
  <button type="button" id="registerButton">Regístrase</button>
</form>
</div>
</div>
</main>
<script src="{% static 'js/login.js' %}"></script>
<script src="{% static 'js/login2.js' %}"></script>
</body>
</html>
```

```
def registro_view(request):
    if request.method == 'POST':
        form = RegistroForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)
            user.set_password(form.cleaned_data['password'])
            user.save()
            login(request, user)
            return redirect('login')
        else:
            form = RegistroForm()
            return render(request, 'login.html', {'form': form})
```

Figura 4. Código para guardar el usuario y contraseña en BD.



Después de que el usuario inicia sesión, se muestra el frame de inicio (**Figura 5**) de la aplicación MAPIM. En la parte superior, la barra de navegación contiene cuatro secciones principales: Inicio, Escaneo, Resultado y Historial, lo que facilita el acceso rápido a las diferentes funcionalidades de la aplicación. En la esquina superior derecha, el usuario puede ver su perfil o cerrar sesión, manteniendo la interfaz limpia y fácil de usar.

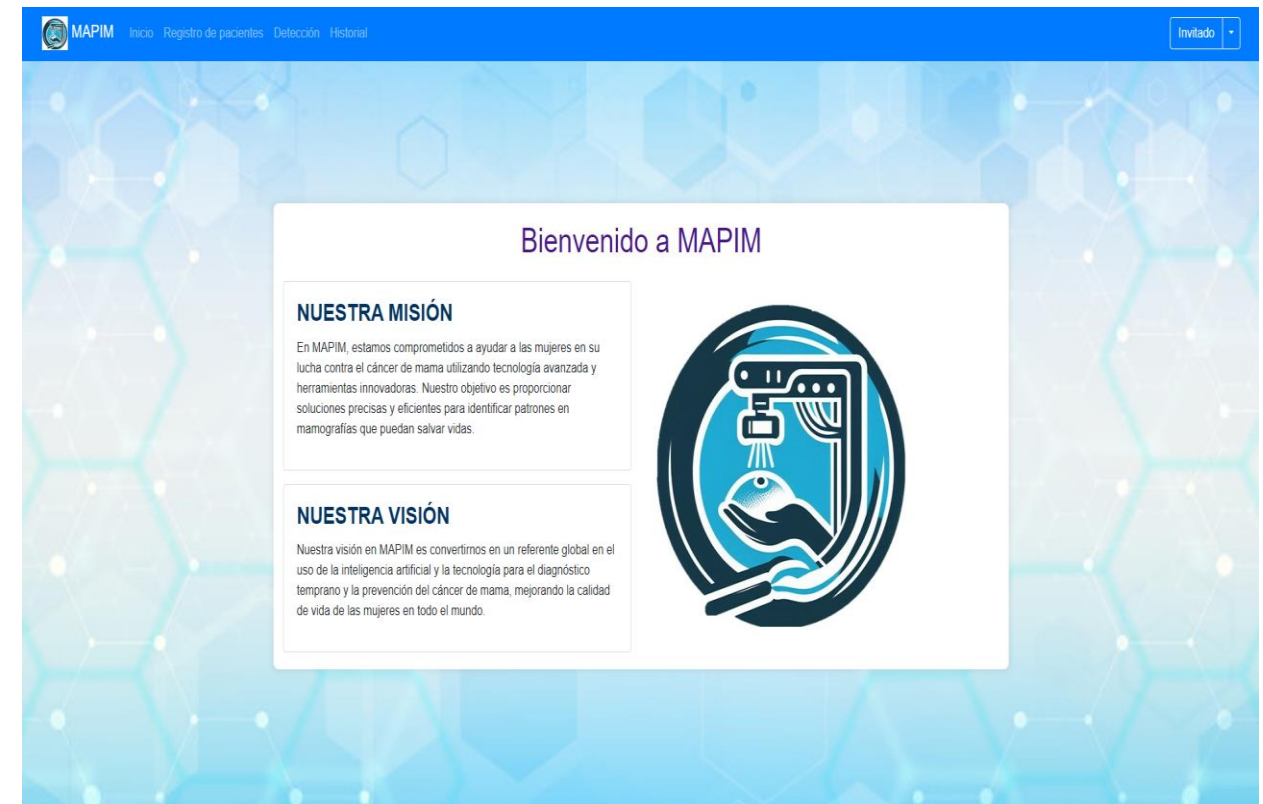


Figura 5. Frame de Inicio



Tras elegir la opción: Registro de pacientes, aparecerá el siguiente frame (**Figura 6**), donde se tendrá que ingresar los datos necesarios para registrar al paciente y guardarlo para realizara el análisis de escaneo.

The screenshot shows the 'Registro de Paciente' (Patient Registration) form within the MAPIM application. The form is displayed on a blue background with a hexagonal pattern. The form itself is white and contains the following fields and labels:

- DNI:** A single-line text input field with the placeholder text 'Ingrese su DNI'.
- Nombre:** A single-line text input field with the placeholder text 'Ingrese su Nombre'.
- Apellido Paterno:** A single-line text input field with the placeholder text 'Ingrese su Apellido Paterno'.
- Apellido Materno:** A single-line text input field with the placeholder text 'Ingrese su Apellido Materno'.
- Número de Contacto:** A single-line text input field with the placeholder text 'Ingrese su Número de Contacto'.
- Dirección:** A single-line text input field with the placeholder text 'Ingrese su Dirección'.
- Fecha de Nacimiento:** A date input field with the placeholder text 'dd/mm/aaaa' and a calendar icon on the right.
- Registrar:** A blue button with white text located at the bottom of the form.

The top of the application window shows a blue header with the MAPIM logo, navigation links (Inicio, Registro de pacientes, Detección, Historial), and a 'Iniciar sesión' button.

Figura 6. Frame de registro del pacientes



En la figura 7, se muestra en código la lógica del procedimiento de los datos que se deben llenar. Aquí se encuentra el botón de registrar al paciente.

```
<body>
<div class="container mt-2">
  <h1 class="text-center mb-4">Registro de Paciente</h1>
  <div class="container-form">
    <form method="POST" action="{% url 'registrar_paciente' %}" enctype="multipart/form-data" onsubmit="return validateForm()"
      (% csrf_token %)>

      <div class="form-group">
        <label for="dni">DNI:</label>
        <input type="text" id="dni" name="dni" class="form-input" placeholder="Ingrese su DNI" value="{% dni %}" maxlength="8" pattern="^(?!([0-9])\1{7})([0-9]{8})$" title="Debe ser un número de 8 dígitos y no todos los dígitos pueden ser iguales">
      </div>

      <div class="form-row">
        <div class="form-group">
          <label for="nombres">Nombre:</label>
          <input type="text" id="nombres" name="nombres" class="form-input" placeholder="Ingrese su Nombre" value="{% nombres %}" minlength="2" required title="Debe contener al menos dos letras">
        </div>

        <div class="form-group">
          <label for="apellido_paterno">Apellido Paterno:</label>
          <input type="text" id="apellido_paterno" name="apellido_paterno" class="form-input" placeholder="Ingrese su Apellido Paterno" value="{% apellido_paterno %}" minlength="2" required title="Debe contener al menos dos letras">
        </div>
      </div>

      <div class="form-row">
        <div class="form-group">
          <label for="apellido_materno">Apellido Materno:</label>
          <input type="text" id="apellido_materno" name="apellido_materno" class="form-input" placeholder="Ingrese su Apellido Materno" value="{% apellido_materno %}">
        </div>

        <div class="form-group">
          <label for="numero_contacto">Número de Contacto:</label>
          <input type="text" id="numero_contacto" name="numero_contacto" class="form-input" placeholder="Ingrese su Número de Contacto" value="{% numero_contacto %}" pattern="^[0-9]{9}$" maxlength="9" title="Debe comenzar con '9' y tener 9 dígitos">
        </div>
      </div>

      <div class="form-group">
        <label for="direccion">Dirección:</label>
        <input type="text" id="direccion" name="direccion" class="form-input" placeholder="Ingrese su Dirección" value="{% direccion %}">
      </div>

      <div class="form-group">
        <label for="fecha_nacimiento">Fecha de Nacimiento:</label>
        <input type="date" id="fecha_nacimiento" name="fecha_nacimiento" class="form-input" value="{% fecha_nacimiento %}">
      </div>

      <button type="submit" class="btn-submit">Registrar</button>
    </form>
  </div>
</div>
</body>
```

Figura 7. Código para ingresar los datos y registrar al paciente.



En la figura 8, se muestra en código de guardar al paciente.

```
def registrar_paciente(request):
    if request.method == 'POST':
        dni = request.POST.get('dni')
        nombres = request.POST.get('nombres')
        apellido_paterno = request.POST.get('apellido_paterno')
        apellido_materno = request.POST.get('apellido_materno')
        fecha_nacimiento = request.POST.get('fecha_nacimiento')
        direccion = request.POST.get('direccion')
        numero_contacto = request.POST.get('numero_contacto')
        usuario_id = request.POST.get('usuario_id')

        # Retrieve the user if it exists, otherwise set it to None
        usuario = None
        if usuario_id:
            try:
                usuario = Usuario.objects.get(id=usuario_id)
            except Usuario.DoesNotExist:
                messages.warning(request, "Usuario no encontrado, se guardará sin usuario asignado.")
                usuario = None # No user found, proceed without

        # Create or update patient record
        paciente, created = Paciente.objects.update_or_create(
            dni=dni,
            defaults={
                'nombres': nombres,
                'apellido_paterno': apellido_paterno,
                'apellido_materno': apellido_materno,
                'fecha_nacimiento': fecha_nacimiento,
                'direccion': direccion,
                'numero_contacto': numero_contacto,
                'usuario': usuario,
            }
        )

        if created:
            messages.success(request, "Paciente registrado exitosamente.")
        else:
            messages.info(request, "Información del paciente actualizada.")

        return redirect('escaneo')

    return render(request, 'escaneo.html')
```

Figura 8. Código para guardar al paciente



En la figura 9, se observa el frame de Detección, en el cual ya registrado el paciente solo se Buscara por su DNI, tras ingresar el DNI saldrá el nombre del paciente y se seleccionara la imagen mamográfica que desea escanear, haciendo click en el botón realizar escaneo escanea donde en la parte de abajo saldrá la detección y precisión, también en la parte de abajo se encuentra un botón para guardar los registros de la detección.

Figura 9. Frame Detección



En la figura 10, se muestra en código la lógica para mostrar el resultado del análisis de la imagen.

```
def procesar_imagen(request):  
    if request.method == 'POST' and request.FILES.get('imagen'):  
        image_file = request.FILES['imagen']  
        image_path = default_storage.save('tmp/' + image_file.name, ContentFile(image_file.read()))  
  
        # Ejecutar la inferencia en la imagen  
        output_data = run_inference(default_storage.path(image_path))  
        predicted_class = np.argmax(output_data[0])  
        probability = np.max(output_data[0]) * 100  
  
        # Mapeo de la predicción a las etiquetas  
        result_map = {  
            0: "Maligno",  
            1: "Benigno",  
            2: "Desconocida"  
        }  
        result = result_map.get(predicted_class, "Resultado no válido")  
  
        # Preparar respuesta en JSON  
        response_data = {  
            'prediction': result,  
            'probability': round(probability, 2)  
        }  
        return JsonResponse(response_data)  
  
    return JsonResponse({'error': 'Método no permitido o archivo no recibido'}, status=400)
```

Figura 10. Código para generar el resultado del análisis de la imagen



En la figura 11, se muestra en código de guardado del resultado de la imagen.

```
def guardar_resultado(request):
    if request.method == 'POST':
        try:
            data = json.loads(request.body)
            dni = data.get('dni')
            imagen_base64 = data.get('imagen')
            resultado = data.get('resultado')
            precision = data.get('precision')

            if not dni or not imagen_base64 or not resultado or not precision:
                return JsonResponse({'error': 'Faltan datos en la solicitud'}, status=400)

            # Crea la detección, y `fecha` se registrará automáticamente con la fecha actual
            Deteccion.objects.create(
                dni_paciente=dni,
                imagen=imagen_base64,
                resultado=resultado,
                precision=precision,
                # Si quieres especificar manualmente, usa: fecha=timezone.now()
            )

            return JsonResponse({'success': 'Detección registrada exitosamente'})

        except json.JSONDecodeError:
            return JsonResponse({'error': 'Formato de JSON inválido'}, status=400)
        except Exception as e:
            print("Error al procesar la solicitud:", e)
            return JsonResponse({'error': 'Error al registrar la detección.'}, status=500)

    return JsonResponse({'error': 'Método no permitido'}, status=405)
```

Figura 11. Código para guardar resultado



En la figura 12, se observa el frame de historial donde se podrá buscar por el DNI y recargar la pagina.

 MAPIM

[Inicio](#) [Registro de pacientes](#) [Detección](#) [Historial](#)

Iniciar sesión

Historial de Escaneos

Buscar por DNI Recargar

DNI	Nombres	Apellidos	Fecha	Tipo	Accion
71277447	Jhoan Ariel	Rivas Yzquierdo	09/11/2024	Maligno	<button>Eliminar</button>
60719252	Vanesa	Sanches Neira	09/11/2024	Benigno	<button>Eliminar</button>
46199815	Sergio	Guzman Fernandez	09/11/2024	Benigno	<button>Eliminar</button>
46199815	Sergio	Guzman Fernandez	09/11/2024	Benigno	<button>Eliminar</button>
46199815	Sergio	Guzman Fernandez	09/11/2024	Benigno	<button>Eliminar</button>

Figura 12. Frame Historial



En la figura 13 se muestra en código la lógica para mostrar los historiales.

```
def historial(request):
    dni = request.GET.get('dni')
    user_id = request.GET.get('user_id')

    # Filtrar registros de detección de acuerdo con los filtros proporcionados
    if dni and user_id:
        historiales = Deteccion.objects.filter(dni_paciente=dni, usuario_id=user_id)
    elif dni:
        historiales = Deteccion.objects.filter(dni_paciente=dni)
    elif user_id:
        historiales = Deteccion.objects.filter(usuario_id=user_id)
    else:
        historiales = Deteccion.objects.all()

    # Añadir nombres y apellidos de cada paciente si existe en la tabla de pacientes
    for historial in historiales:
        if historial.dni_paciente: # Verificar si hay un dni_paciente presente
            try:
                paciente = Paciente.objects.get(dni=historial.dni_paciente)
                historial.nombres = paciente.nombres
                historial.apellidos = f"{paciente.apellido_paterno} {paciente.apellido_materno}"
            except Paciente.DoesNotExist:
                historial.nombres = "-"
                historial.apellidos = "-"
        else:
            historial.nombres = "-"
            historial.apellidos = "-"

    return render(request, 'historial.html', {'historiales': historiales})
```

Figura 13 Código para generar el historial.



Universidad
César Vallejo